Computational Science (joint degree)

Vrije Universiteit Amsterdam - Faculty of Science - Computational Science (joint degree) - 2025-2026

Computational Science seeks to explain the complex world we live in using modelling and simulations. By collecting data from various sources and creating computer models, computational scientists can make predictions for problems such as how to influence the flow of traffic, how an epidemic will spread or what the probability is that individuals in society will become addicted to drugs.

Computational Science is a joint degree master's programme of University of Amsterdam and VU Amsterdam. For further information about the programme go to the UvA Course Catalogue:

https://studiegids.uva.nl/xmlpages/page/2024-2025/zoek-opleiding/opleiding/8217

Info

Level	Master
Language	English
Duration	2 years
Type	Full Time
Credits	120 EC
Faculty	Faculty of Science

Joint degree
More information about this programme

Computational Science JD Optional Courses taught at VU 4
Courses 4
Accelerator-Centric Computing Ecosystems 4
Algorithms in Sequence Analysis 5
Bioinformatics for Translational Medicine 7
Biosystems Data Analysis 9
Data Mining Techniques 10
Distributed Algorithms 11
Distributed Systems 13
Evolutionary Computing 15
Experimental Design and Data Analysis 16
From Physics to Physiology 17
Fundamentals of Bioinformatics 18
High Performance Computing and Big Data 20
Machine Learning for the Quantified Self 20
Parallel Programming Practical 22
Performance of Networked Systems 23
Programming Large-Scale Parallel Systems 24

Computational Science JD Optional Courses taught at VU

Courses

Course Name	Period	Credits	Code
Evolutionary Computing	P1	6EC	X_400111
From Physics to Physiology	P1	6EC	X_430112
Fundamentals of Bioinformatics	P1	6EC	X_405052
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Algorithms in Sequence Analysis	P2	6EC	X_405050
Distributed Systems	P2	6EC	X_400130
Performance of Networked Systems	P2	6EC	X_405105
Biosystems Data Analysis	P3	6EC	XM_0078
High Performance Computing and Big Data	P3	6EC	XMU_40013
Parallel Programming Practical	P3	6EC	X_400162
Experimental Design and Data Analysis	P4	6EC	X_405078
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Bioinformatics for Translational Medicine	P5	6EC	X_405092
Data Mining Techniques	P5	6EC	X_400108
Distributed Algorithms	P5	6EC	X_400211
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Courses

Accelerator-Centric Computing Ecosystems

Course Code	XM_0171
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. T. De Matteis
Examiner	dr. T. De Matteis
Teaching Staff	dr. D. Bonetta, dr. T. De Matteis
Teaching method(s)	Seminar, Lecture

Course Objective

With the end of Moore's law, the need for continuing performance scaling has led to the emergence of heterogeneous computing systems infrastructures. Alongside traditional multicore CPUs, we now see GPUs (Graphic Processing Units), FPGAs (Field Programmable Gate Arrays), and other domain-specific architectures (such as DPUs and TPUs) being commonly offered as accelerators to speed up part of applications. Each of these devices has different performance, power requirements, and parallelism opportunities that must be understood to leverage the diversity of modern and future ICT infrastructure.

This course aims to equip students with an in-depth understanding of accelerated computing systems. This involves discussing diverse computing architectures, including alternatives to traditional ("von Neumann") computer organization, and analyzing their different programming models and optimization techniques, developing a comprehensive understanding of the intricacies and opportunities of each presented approach.

After completing this course, you will:

- Understand diverse computing architectures (e.g., GPUs, FPGAs, and emerging paradigms) and relate them to classical computer organization [Knowledge and understanding]
- Acquire a comprehensive knowledge of the parallel programming models that characterize different compute

accelerators and optimization techniques for diverse workloads. [Knowledge and understanding]
- Develop the ability to design, analyze, and implement parallel algorithms for specific computing architectures [Apply Knowledge]
- Evaluate and make informed decisions about the selection and optimization of algorithms for diverse computing architectures, considering factors such as performance, energy efficiency, and scalability [Apply Knowledge] [Making judgements]
- Effectively communicate technical concepts related to advanced computing systems [Communication]
- Develop learning skills to stay informed about and critically analyze emerging technologies in advanced computing systems [Lifelong learning skills]

Course Content

The course will consist of weekly lectures, lab sessions, and practical programming assignments.

We will start by discussing the challenges faced by the computing community, such as the end of Dennard scaling and Moore's law, as well as the emergence of new computational-hungry domains – e.g., AI. We will discuss how these challenges have led to the development and diffusion of heterogeneous computing systems. We will introduce general-purpose graphics accelerators (GPGPUs) using, for example, NVidia's programming model CUDA. We will present dataflow architectures (particularly FPGAs) as an alternative to classical von Neumann architectures. We will discuss their programming challenges and how to develop efficient FPGA programs using High-Level Synthesis tools. Finally, we will conclude by discussing novel/research computing architectures and the current research directions in this field.

Additional Information Teaching Methods

2 Lectures per week, Practical coding group assignments with 1 lab session per week for training/QA (deliverables: code and report).

Method of Assessment

There are two components in this course: Exam (50% of the final grade) and Practical Assignments (50% of the final grade). Resit is possible only for the exam. Exam and practical assignments must be passed separately to pass this course.

Practical assignments are done in small teams. Students are graded on the completion of the assignment and their understanding of the proposed solution (based on the oral discussion/interview and report/code submitted).

[FRAUD] The course is made fraud-resistant by having specialized group projects with interviews to assess the knowledge of the students. Furthermore, the exam is taken individually in person on campus.

Please note that your grade will be registered as follows:

- If you did not submit any assignments and did not participate in the exam, the result will be registered as No Show (NS).
- If you did not meet the minimum grade requirements for the assignments or the exam, but your weighted average is >= 5.5, your grade is NVD.
- Otherwise, your final grade is your weighted average of the two components, rounded to half points except for 5.5.
- There is no resit opportunity for the practical assignments. Results for practical assignments and for the exam can not be transferred from previous years.

Literature

The course does not follow a specific textbook.

Slides and further online reading material/links will be provided via Canvas.

Additional Information Target Audience

MSc Computer Science

Recommended background knowledge

Computer organization (CPU caches, instructions, multicores), knowledge of C/C++, and parallel programming. For practical assignments: Linux development environment - gcc, make, gdb, ssh.

Algorithms in Sequence Analysis

Course Code	X_405050
Credits	6
Period	P2
Course Level	600
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	R. Kumar
Examiner	dr. S. Pissis
Teaching Staff	dr. J.M. Urtasun Elizari, dr. ir. K.A. Feenstra, dr. S. Pissis, prof. dr. J. Heringa, dr. Z. Işik MSc, H. Verbeek MSc
Teaching method(s)	Computer lab, Lecture

Course Objective

Have you ever wondered how we can track a gene across 3 billion years of evolution? Or how you can use the genome information of a given cancer patient to find out what may be wrong? Sequence alignment can be used to compare genomes, genes or proteins from bacteria all the way to humans, while further derived algorithms may be employed to make a phylogeny (to find out about evolutionary relationships), find a functional motif in a protein sequence, or a viral sequence in a genome. In this course we focus on the most important algorithms for biological sequence analysis that can be applied to real scientific problems in biology.

After completing the course successfully,

- You can demonstrate in-depth knowledge about the theory of sequence analysis methods.
- You know how to implement several of the most important sequence alignment and analysis algorithms.
- You understand the mathematical formulation behind sequence analysis algorithms.
- You will have understanding and skills to apply sequence analysis algorithms to protein and DNA sequences.
- You will have hands-on experience in tackling biological problems using sequence analysis algorithms, including the statistical framework of HMMs and algorithms used in genome sequencing and analysis.
- You can decide which algorithm is best suited for a particular biological sequence analysis problem.
- You can read through scientific and technical literature and learn to translate algorithms described in text and formal mathematical notation into computer code.

Course Content

Theory:

- Dynamic programming
- database searching
- pairwise and multiple alignment
- probabilistic methods including HMMs
- pattern matching
- entropy measures
- evolutionary models
- and phylogeny

Practical:

- Programming (in Python) an alignment algorithm based on dynamic programming;
- Aligning sequencing data from tumors to the human genome and analysing structural variants;
- Programming (in Python) an implementation of HMMs and using it to predict protein domain structure.

Additional Information Teaching Methods

13 lectures: 2 two-hour lectures per week.
13 computer practicals and associated assignments: 2 two-hour hands-on sessions per week.

Method of Assessment

The final grade for this course will consist of 50% practical work (see above) and 50% theoretical assessment. (assessment of practical assignments may be adjusted based on ongoing developments in generative language models)

The theoretical assessment will be an oral and/or written exam (depending on number of students).

Further assessment and grading details will be posted on Canvas (resits and compensation rules).

Entry Requirements

Bachelor in any science discipline (including medicine).
Basic programming skills (Python) and an interest in biological problems.

Literature

Course material on Canvas.

Books: Durbin, R., Eddy, S.R., Krogh, A., Mitchison, G.. Biological Sequence Analysis. Cambridge University Press, 1998, 350 pp., ISBN 0521629713.

Recommended reading: Marketa Zvelebil and Jeremy O. Baum Understanding Bioinformatics. Garland Science 2008. ISBN-10: 0-8153-4024-9

Additional Information Target Audience

M Bioinformatics and Systems Biology
M Biomolecular Sciences
M Artificial Intelligence
M Computational Science

Additional Information

BYOD policy (Bring Your Own Device)

We expect students in this course to use their own laptop. This laptop should at the very least support an SSH client, for remote shell access to the VU Linux servers. Ideally, this laptop supports a command line shell, Python 3 and a text editor with syntax highlighting -- either standalone (e.g. Atom or Sublime Text) or as part of a simple IDE (e.g. Spyder). As such, we recommend the Anaconda python distribution regardless of operating system, along with PuTTy or PowerShell for Windows users specifically.

The course is taught in English.

Bioinformatics for Translational Medicine

Course Code	X_405092
Credits	6
Period	P5
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J.M. Urtasun Elizari
Examiner	dr. E. Bosdriesz
Teaching Staff	dr. E. Bosdriesz, dr. J.M. Urtasun Elizari
Teaching method(s)	Practical, Lecture

Course Objective

Observations from biological high-throughput experiments allow us to improve diagnosis and give a personalized treatment plan for patients. However, integrating data from several sources and using this data for predictions is non-trivial.

This is a theoretical and practical Bioinformatics course on computational methods for Translational Medicine; we will focus on Bioinformatics methods that are used to predict the clinical outcome for patients and analysis methods to obtain deeper understanding of complex diseases, by combining data from various high-throughput experiments such as proteomics, microarrays and next-generation sequencing as well as existing biological databases.

After completing the course successfully,

- You can interpret research results in the context of biology
- You will be aware of Bioinformatics methods that are applicable to the area of Translational Medicine
- You can combine these methods into a creative solution using large-scale biological experiments
- You will have hands-on experience in handling large biological datasets, and will understand the complexity of the biological data both from high-throughput experiments and existing biological databases

- You will be able to work as a team to execute a large, complex biomedical bioinformatics project, and report on the results
- You will be familiar with a few in-depth research topics that lie within the expertise area of several (Bioinformatics) researchers, among others from the VU, UvA, AUMC and NKI

Course Content

Theory

- Computational analysis of molecular profiling techniques, such as WGS, WES, proteomics, RNA sequencing, and arrayCGH.
- Computational methods applied to these data types, such as: machine learning, normalisation, regularization, feature selection, classification, read mapping, clustering.
- All data analysis is relevant in a clinical setting, for diagnosis, treatment decisions or biomarker discovery.

Practical

- A large assignment for which you have to build a classifier based on molecular profiling data (the precise data-set is to be determined). You need to hand in predictions, write a paper and give a presentation. Note that this is a group project, and to participate in this practical you need to have obtained at least 12 ECT from Master-level courses.
- A smaller assignment that involves analysing and understanding molecular profiling data of another modality than the large classifier assignment.

Additional Information Teaching Methods

- 13 lectures (2 two-hour lectures per week)
- 12 computer practicals (2 two-hour sessions per week)

Method of Assessment

The final grade for this course will consist of 50% practical work (see above) and 50% theoretical assessment.

Practical assessment (50%):

- Classifier assignment (45%)
- Other data modality assignment (5%)

(assessment of practical assignments may be adjusted based on ongoing developments in generative language models)

Theoretical assessment (50%):

- Oral or written exam (depending on number of course students)
- The exam is based on a selection of 8-10 scientific papers in the field of Bioinformatics & Translational Medicine.

Entry Requirements

Importantly: to participate in the group assignment, you need to have obtained a minimum of 12 ECT of Master level courses. In addition, some basic programming skills (either in R or Python) are required, as well as some basic knowledge on molecular biology. If you are not following the Master Bioinformatics and Systems Biology, it is advisable to first follow the Master course "Fundamentals of Bioinformatics".

Literature

- Course material on Canvas, including slides and recordings of lectures.
- 8-10 scientific papers are provided, and make up the course syllabus.

Additional Information Target Audience

M Artificial Intelligence
M Bioinformatics and Systems Biology
M Biomolecular Sciences
M Oncology
M Computational Science

Additional Information

The course is taught in English.

- Compulsory course for students in Master Bioinformatics and Systems Biology
- Optional course for students with a Bachelor in Physics, Chemistry, Mathematics, Computer Science, Biology, or Biomedical Sciences (see requirements below).

Recommended background knowledge

If you are not following the Master Bioinformatics and Systems Biology, it is advisable to first follow the Master course "Fundamentals of Bioinformatics".

Biosystems Data Analysis

Course Code	XM_0078
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J.A. Westerhuis
Examiner	dr. J.A. Westerhuis
Teaching Staff	dr. D. Molenaar, dr. J.A. Westerhuis, dr. G. Stolf Jeuken
Teaching method(s)	Computer lab, Written partial exam, Lecture

Course Objective

The amount of omics data being gathered, analysed, integrated and interpreted increases every year. The number of algorithms, packages and webservices also increases very fast. With the high number of omics features e.g. genes, proteins or metabolites measured, the chances of drawing incorrect conclusions are rather high.

In this course we will discuss properties of the measured data, assumptions of the data analysis methods being used, data preprocessing and pretreatment and validation of the conclusions and interpretations.

Upon completion of this course, students will have reached the following goals

1. Students are able to use R for multivariate analysis of omics data. [KBB 4]
2. Students understand linear algebra concepts that are used in data analysis methods. [KBB 5]
3. Students are able to apply data preprocessing methods (variance stabilization, scaling, transformation), explorative data analysis methods (PCA, Clustering) and supervised data analysis (PCDA, ASCA, Random Forest). [KAR 1]
4. Students are able to evaluate which preprocessing methods, and which data analysis method to use for a given problem. [KAR 5]
5. Students are able to analyse omics data and interpret the results. [KAR 1]
6. Students are able to evaluate the quality of the data analysis models using cross validation and permutation methods. [KAR 4]
7. Students understand how the data analysis methods work theoretically. [KBB5]
8. Students are able to critically review data analysis applications in which the methods taught methods have been used. [KAR 4]

Course Content

In the analysis of biochemical systems, the tendency is to measure more and more of just a few samples, leading to complex multivariate data sets. Multivariate data analysis methods are often used to explore such sets.

This course covers a broad range of multivariate data analysis methods, for data preprocessing (e.g. variance stabilization), for exploration analysis (PCA, Clustering, nonlinear mapping), supervised analysis (ASCA, Classification, Random Forest). Furthermore, validation techniques such as cross validation and permutation testing are introduced and the interpretation of selected features in terms of function and networks is discussed. The course starts with an introduction on the properties of the different types of functional genomics data.

The main goal of this course is to teach students the theoretical properties of the data analysis methods and how to interpret the results of the multivariate methods and how this relates to the biological problem that is studied.

Additional Information Teaching Methods

- 10 lectures (2h each)
- 10 laptop seminars (4h each)
- Self-study for each of the 10 topics

Method of Assessment

There is a data analysis project in which students apply several methods to a dataset after which they have to interpret the results with respect to the statistical properties and the biological properties of the results. There is a theoretical test in which open questions are asked on the data analysis methods, their properties and especially on how to interpret their results. For the final grade, the weights are 1/3 and 2/3 for the project and the theoretical test, respectively. Further assessment and grading details will be posted on Canvas (resits and compensation rules).

Entry Requirements

Bachelor in any science discipline, with an interest in biological data analysis. Basic level of linear algebra required. When in doubt please contact the coordinator.

Literature

Literature:

- Will be provided through Canvas.

Software:

- R.

Additional Information Target Audience

Admission to the course will depend on capacity, total number of applications, date of registration and background of the individual student. If the number of applications exceeds the capacity of the course, students may have to be selected and priority will be given in the following order:

- First-year students of the master Bioinformatics and Systems Biology (JD UvA and VU).
- Second-year students of the master Bioinformatics and Systems Biology (JD UvA and VU).
- Students of the master Computational Science (JD UvA and VU).
- Students of other master programmes.

Recommended background knowledge

Linear Algebra, Statistics with R

Data Mining Techniques

Course Code	X_400108
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	dr. S. Yu, prof. dr. M. Hoogendoorn, B. Borsboom
Teaching method(s)	Computer lab, Lecture

Course Objective

The aim of the course is that students acquire data mining knowledge and skills that they can apply in a business environment. More precisely, the following learning goals are distinguished:

1. Understand the data mining process
2. Understand exploratory data analysis
3. Understand feature engineering
4. Understand classification techniques
5. Understand regression techniques

6. Understand association rules and recommender systems
7. Understand big data engineering and big data infrastructures
8. Understand current state of the art in data mining
9. Understand ethical concerns and evaluation of results
10. Learning to argue for choices (rationale)

The course if focused on the Dublin descriptors Applying Knowledge and Understanding (since the aim is to improve the practical skills); Making judgements (which technique is appropriate, how to best apply Data Mining for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

How the aims are to be achieved: Students will acquire knowledge and skills mainly through the following: an overview of the most common data mining algorithms and techniques (in lectures), a survey of typical and interesting data mining applications, and practical assignments to gain "hands on" experience. The application of skills in a business environment will be simulated through various assignments of the course.

Course Content

The course is intended to introduce Data Mining Techniques to students that are new to the field as well as to more experienced students. The main aim is to gain a more practical perspective towards Data Mining Techniques/Machine Learning. Lectures will cover more basic things for those new to the field (general introduction into Data Mining, classical algorithms such as decision trees, association rules, neural networks, ensemble learning, etc.) and on top will discuss advanced topics including deep learning, recommender systems, big data infrastructures, and text mining. A number of successful applications in the area will also be discussed. In addition to lectures, there will be an extensive practical part, where students will experiment with various data mining algorithms and data sets. The grade for the course will be based on these practical assignments.

Additional Information Teaching Methods

Lectures (h) and practical sessions (pra). Lectures are planned to be interactive: there will be small questions, etc.

Method of Assessment

Practical assignments. There will be two assignments done in groups of three. For the first assignment there is a choice: going for a basic version of the assignment (suited for those new to the domain) or a more advanced one (for students with more experience). The second assignment is the same for all and will involve an in-class Kaggle competition. The first assignment counts for 40% and the second for 60%.

An interview will take place on an individual student level to make sure that (1) each student understands the content of the graded material, and (2) has made a sufficient contribution to the group work. Only when this check is passed will a final grade be given. Furthermore, the grade of both assignments needs to be sufficient to pass the course. Finally, attendance of the final lecture (in which the final ranking of the Kaggle competition is announced and selected groups of students will be asked to present their solution) is mandatory as it contributes to the learning objectives of the course.

There is an option to resit one of the assignments when the assignment has been submitted but an insufficient grade is obtained (only when the other assignment grade is sufficient). This involves improving your assignment to at least a sufficient level within two weeks after having received your grade for the second assignment. The maximum grade of the resit is a 6.0.

Literature

Optional: Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. Data Mining: Practical Machine Learning Tools and Techniques (Fourth Edition). Morgan Kaufmann, 2016 978-0128042915.

Additional Information Target Audience

Master Artificial Intelligence, Master Computer Science, Master Bioinformatics and Systems Biology, Master Business Analytics

Recommended background knowledge

1. Statistics
2. Programming Skills

Distributed Algorithms

Course Code	X_400211

Credits	6
Period	P5
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. W.J. Fokkink
Examiner	prof. dr. W.J. Fokkink
Teaching Staff	prof. dr. W.J. Fokkink, dr. Y. Nazari MSc
Teaching method(s)	Seminar, Lecture

Course Objective

The main objective is to develop an algorithmic frame of mind for solving fundamental problems in distributed computing, by obtaining insight into concurrency concepts. The course offers a bird's-eye view of a wide range of algorithms for basic and important challenges in distributed systems.

A key characteristic of the course is that correctness arguments and complexity calculations of distributed algorithms are provided in an intuitive fashion and by means of examples and exercises.

At the end of the course, the student will:

- be acquainted with fundamental aspects of distributed algorithms, notably logical clocks, wave algorithms, snapshots, rollback recovery, consensus, and self-stabilization; (knowledge and understanding)
- be able to apply algorithms for termination and deadlock detection, routing, leader election, and mutual exclusion; (applying knowledge and understanding)
- have a grasp of impossibility results regarding anonymous networks and networks in which nodes can crash; (knowledge and understanding)
- be aware of certain attacks, data structures (notably Merkle trees), and key exchange and authentication protocols to safeguard the security of distributed systems; (knowledge and understanding)
- have a clear understanding of advanced protocols for distributed transactions, cryptocurrencies, and quantum cryptography. (knowledge and understanding)

Course Content

The following topics are treated in the course: Logical clocks, snapshots, graph traversal, termination detection, garbage collection, deadlock detection, routing, election, minimal spanning trees, anonymous networks, checkpointing, fault tolerance, failure detection, consensus, mutual exclusion, self-stabilization, distributed transactions, dynamic networks, key exchange, quantum cryptography, authentication, digital signatures, blockchains.

Additional Information Teaching Methods

Lectures: 4 hours per week. Exercise classes: 2 hours per week.

Method of Assessment

Written exam. At the exam, a physical copy or print-out of the textbook can be used. All books will be checked by the lecturer at the exam.

Literature

W.J. Fokkink, Distributed Algorithms: An Intuitive Approach (forthcoming 3rd edition, draft), MIT Press, 2025.

A pdf of the textbook is made available on the Canvas page.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Econometrics and Operations Research

Recommended background knowledge

Data Structures & Algorithms.

Distributed Systems

Course Code	X_400130
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	ir. J.J.R. Donkervliet MSc
Examiner	prof. dr. ir. A. Iosup
Teaching Staff	ir. S. Talluri, prof. dr. ir. A. Iosup, dr. T. De Matteis, ir. J.J.R. Donkervliet MSc
Teaching method(s)	Seminar, Lecture

Course Objective

By taking this course, you will learn to:

1. Explain the basic concepts, objectives, and functions of distributed computing (eco)systems, e.g., communication, resource management and scheduling, data consistency, fault-tolerance, and performance. (Knowledge and understanding)
2. Compare the basic characteristics of distributed computing with other computing paradigms (i.e., centralized, parallel). (Applying knowledge and understanding) (Making judgements)
3. Identify proficiently the different flavors of modern distributed ecosystems (i.e., peer-to-peer systems, cluster computing, grid computing, cloud computing, datacenters, distributed HPC, SDN, Big Data systems, IoT systems). (Knowledge and understanding) (Applying knowledge and understanding) (Making judgements)
4. Analyze proficiently the trade-offs inherent in the design of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
5. Design your portfolio distributed system, with many basic and some complex operations of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
6. Implement and deploy your portfolio distributed-system. (Applying knowledge and understanding) (Making judgements)
7. Analyze your portfolio distributed-system. (Communication) (Lifelong learning skills)

Course Content

This course focuses on distributed computing systems and ecosystems. In general, debugging and tuning existing systems and ecosystems, and designing, implementing, and analyzing new distributed computing systems remain vital and challenging for both industry and academia.

Starting with the mid-1990s, computing is undergoing a revolution, in which collections of independent computers appear to users as a single, albeit distributed, computing system. Motivated by the advent of the Internet, the increase in the computation capacity of consumer computers, the commoditization of server-grade machines, energy constraints, etc., the distributed computing paradigm has permeated all fields using computers. Current distributed computing applications range from social networks to banking, from peer-to-peer file-sharing to high-performance computing used in research, from massively multiplayer online games to business-critical workloads, etc. Important advances have helped to fuse heterogeneous resources into truly global distributed systems and ecosystems, for example in scientific computing, where distributed computation is using Big Data and distributed sensors to produce meaningful progress for humankind. We will focus in this course on a number of these modern examples of distributed computing systems.

Although so many distributed systems and ecosystems already exist, the list of conceptual and technical challenges they pose is long. Depending on requirements, even trivial communication between nodes of the distributed system can be challenging. The failure of a single node, or sometimes even a performance hiccup, can bring an entire system down with it, other nodes or entire other systems may also crash, experiencing correlated and catastrophic failures. Data consistency and coordinating nodes remain important challenges made worse by the large scale of real-world deployments. Poor resource management and naive scheduling can lead to orders-of-magnitude higher operational costs and consumption of energy that we simply cannot spare. It is not uncommon for a modern distributed system to quickly rise and then fall in popularity, as exemplified by the 2016 example of Pokemon Go. We will present in this course real-world situations where modern distributed systems have behaved poorly.

Addressing these challenges requires unique approaches and concepts.

Separating concerns and breaking down problems into smaller cases often lead to limited success, because many properties of distributed systems can only be achieved end-to-end. Can anyone imagine a perfectly reliable

production pipeline, if even one of its key stages can suffer failures?

Building capability by adding resources is often offset by the distributed nature of the system. Can anyone ignore the physical limitations of communication around the globe? In this course, we will focus on the unique approaches and principles of distributed systems, from specific architectures and communication protocols to specific concepts in resource management and scheduling, data consistency, fault-tolerance, and performance.

Additional Information Teaching Methods

The course uses prof. Iosup's gamification method. The course is taught as a series of lectures, in combination with self-study and design sessions, and with a large practical assignment. The course also has important design and seminar components.

Method of Assessment

Written exam. Report on the practical assignment. In-class assignments include: design, seminar, Q&A. There are other gamification elements.

For a total maximum of 10,000 points (equivalent to a grade of 10), the course activities will be graded as follows:

1. Exam, questions (max 6,000 points)
2. Practical, lab exercise (max 4,000 points)
3. Design and seminar sessions (max 2,000 points)
4. In-class activity: max 50p per session

Other bonuses may be applicable

There is only a resit opportunity for the exam.

Literature

The course uses the following textbook: Maarten van Steen and Andrew S. Tanenbaum, Distributed Systems, 3rd. Ed., online edition, 2017. (free for all) [Online] Available: https://www.distributed-systems.net/index.php/books/distributed-systems-

Recent material is only available as lecture slides and selected publications. The lecture slides also recommend additional literature.

The study guide available on Canvas also indicates other worthwhile sources of information.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Additional Information

This course uses the gamification technique developed by prof. Iosup and his team.

Explanation Canvas

The course guide is mandatory reading for every student prior to any other activity in the course.

Recommended background knowledge

Ability to work in teams for the practical assignment. Ability to design software and to develop code using modern software engineering practices, e.g., setting up your own GitHub repository, co-editing using tools such as Overleaf or Sharelatex, etc., is a big plus. Experience with big data processing, multimedia and online gaming systems, scientific (workflow) computing is also a plus.

Students should have taken standard courses on:

- Computer networks
- Operating systems
- Programming paradigms, in particular OOP and/or actor-based approaches
- Software engineering

Prior experience with Internet, web, distributed, or parallel programming and algorithms is helpful.

Prior experience with operating systems development and analysis, and in general experience with computer

systems courses, is a big plus.

Evolutionary Computing

Course Code	X_400111
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. A.E. Eiben
Examiner	prof. dr. A.E. Eiben
Teaching Staff	prof. dr. A.E. Eiben, mr. J.M. di Matteo, dr. K. da Silva Miras de Araujo
Teaching method(s)	Seminar, Lecture

Course Objective

This course has a threefold objective:

1. (Knowledge & Understanding) To learn about computational methods based on Darwinian principles of evolution.
2. (Applying knowledge and understanding, Communication) To use such methods as problem solvers and as simulation tools; to determine adequate algorithm setups depending on the problem at hand.
3. (Applying knowledge and understanding, Lifelong learning skills) To gain hands-on experience in performing computational experiments with evolutionary algorithms.

Course Content

This course is about constructing, applying and studying algorithms based on the Darwinian evolution theory. Driven by selection (survival of the fittest, mating of the fittest) and randomised reproduction (mutation, recombination), an evolutionary process is being emulated and solutions for a given problem are being "bred". During this course, various flavours within evolutionary computing are treated, including genetic algorithms, evolution strategies, evolutionary programming, genetic programming, differential evolution, particle swarm optimisation. Applications in optimisation, constraint handling, machine learning, and robotics are discussed. Specific subjects handled include: genetic structures (representations), selection techniques, sexual and asexual reproduction operators, (self-)adaptivity and methodological aspects, such as algorithm design & tuning, and performance assessment. Special attention is paid to the field of evolutionary robotics where the bodies (morphology, hardware) and the brains (controller, software) of robots are evolved simultaneously. Hands-on-experience with evolutionary algorithms is gained through a compulsory programming assignment.

Additional Information Teaching Methods

Oral lectures and compulsory Python programming assignment (in teams of 3 to 5). A limited number of highly motivated students can replace the programming assignment by a special research assignment under the personal supervision of the lecturer(s). These research projects aim at publications with the students as first authors of the paper.

Method of Assessment

Written exam and programming assignment (in teams of 3 to 5). To pass the course as a whole, you must pass both the exam and the programming assignment. For the final grade they will count with the same weight:

Grade = 0.5 x Exam_Grade + 0.5 x Assignment_Grade

Note that:

- The exam is multiple choice, taken on the campus in an electronic form. There is a resit in case you cannot attend or fail.
- The programming assignment is graded based on your code, the quality of the solution your algorithm finds, and a short report. **No resit is possible for the programming assignment.**
- If you pass one component (Exam or Assignment), but fail the other, then you can keep the grade of the passed component for next year or later. Then you can pass the course by getting a good grade (> 5.5) for the missing component and using the old grade in the averaging formula.

Literature

The course is based on the textbook ***A.E. Eiben and J.E. Smith, Introduction to Evolutionary Computing, Springer, 2015, 2nd edition, ISBN 978-3-662-44873-1***. The lectures and the book chapters are synchronised to a great extent. In addition, a number of carefully selected papers to provide extra information on specific topics are available via Canvas.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Econometrics and Operations Research
MSc Finance
MSc Business Analytics
MSc Computer Science
MSc Computer Security
MSc Bioinformatics and Systems Biology

Additional Information

Attending the lectures in real life is not mandatory, but highly recommended. Active participation in the team work for the programming assignment is a must!

Explanation Canvas

The complete schedule of the course, organised by the chapters of the textbook, will be made available at the beginning of the course. Students are assumed to consult the course schedule to keep up with the progress, as Canvas rubrics are not used. Video recordings of the lectures made during the COVID period are still available on Canvas for self-study.

Recommended background knowledge

Python programming skills are necessary to do the practical assignment. If you cannot program, you cannot complete this course.

Experimental Design and Data Analysis

Course Code	X_405078
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. E.N. Belitser
Examiner	dr. E.N. Belitser
Teaching Staff	dr. E.N. Belitser
Teaching method(s)	Practical, Lecture

Course Objective

In this course, students will become familiar with the most commonly used experimental designs, regression models, some nonparametric tests and bootstrap methods. By the end of the course, students will be able to:

- Identify and describe some of the most widely used statistical models,
- Understand the basic theory underlying these models,
- Apply these models appropriately in relevant contexts.

Course Content

The EDDA course introduces a range of widely used statistical models and techniques, including paired and independent t-tests, non-parametric rank-based tests, bootstrap and permutation methods, analysis of variance (ANOVA), ANCOVA, lasso, linear regression models, generalized linear and nonlinear models, and survival analysis. Students will learn how to choose appropriate models for various scenarios, estimate model parameters using available data, and validate model fit. For regression models, proper study design is essential to draw sound conclusions; the course covers well-known experimental designs such as completely randomized and randomized block designs, along with their associated ANOVA techniques. A portion of the course focuses on non-parametric and resampling methods, such as the Wilcoxon test (one- and two-sample), Kolmogorov–Smirnov test, rank correlation tests, and both permutation and bootstrap tests. The methods will be illustrated using practical

examples, with an emphasis on real datasets. The statistical software R will be used for data analysis throughout the course.

Additional Information Teaching Methods

Lectures, practical sessions.

Method of Assessment

Course assessment consists of assignments (33.3%) carried out in teamwork during the course and an individual digital test (66.7%) at the end. To pass the course, both components must meet minimum required thresholds. Re-examination policy: if the final result is insufficient, only the individual test may be retaken.

Literature

Literature will be communicated on Canvas.

Additional Information Target Audience

MSc AI, CS

Additional Information

Assignments are to be made using the statistical package R (http://www.r-project.org).

Explanation Canvas

All the information about the course will be available on Canvas.

Recommended background knowledge

- A basic course in statistics of the same level as Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080) is mandatory. All the material and its prerequisites (basic mathematical and programming skills) for these courses will be assumed known in the lectures/assignments/exam of the course EDDA.
- A good background in probability, at least of the same level as in Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080).
- Basic knowledge of the statistical software R and its application to data analysis.

From Physics to Physiology

Course Code	X_430112
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. I.H.M. van Stokkum
Examiner	dr. I.H.M. van Stokkum
Teaching Staff	dr. I.H.M. van Stokkum
Teaching method(s)	Seminar, Study Group, Practical, Written partial exam, Lecture

Course Objective

- To understand fundamental (bio)physical processes and their relevance for physiology.
- To gain knowledge of the application of physical methods in modelling physiological processes
- To gain knowledge of the most important theoretical and practical concepts in modelling and simulation of biomedical processes at different scales.
- To gain experience with and to apply MatLab and Mathematica to acquire, analyse and evaluate biomedical signals and to model and simulate biomedical processes.

Course Content

The course starts with a general overview of the various types of models used to describe biomedical processes by parametric and non-parametric models using linear and non-linear (differential) equations. Some of the

fundamental biophysical principles that have broad relevance in biomedical research will be presented. Practical problems discussed during the lectures will concentrate on modelling the cardiovascular system using mechanical and electrical analogues.

The lectures during the first four weeks will be combined and followed by working groups in which, through relatively simple exercises, experience will be gained of MatLab and Mathematica, two frequently used 4th generation computer languages.
Finally, students will be offered a choice of 1 out of several modelling problems to be solved in small groups, guided by a supervisor. At the end of the course each group will present and discuss their work with all participants and supervisors of the course.

Additional Information Teaching Methods

Lectures, working groups, assignments.

Method of Assessment

Homework assignments (Pass/Fail), report and presentation on modelling problem (40%) and written exam (60%).

Literature

Syllabus.

Additional Information Target Audience

mBMTP, mPhys, mSBI, mComputationalScience

Fundamentals of Bioinformatics

Course Code	X_405052
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J.M. Urtasun Elizari
Examiner	dr. ir. K.A. Feenstra
Teaching Staff	dr. H. Mouhib, dr. J.M. Urtasun Elizari, dr. ir. K.A. Feenstra, dr. D. Molenaar
Teaching method(s)	Seminar, Computer lab, Written partial exam, Lecture

Course Objective

Are you interested in Bioinformatics? Do you want to find out how biology can make an exciting application domain? Or do you want to learn what more you could do with your data, and with less effort? Enter here to start!

Fundamentals of Bioinformatics (FoB) is the starting course of the Bioinformatics master. It aims to give you a broad overview of important topics relevant to the field, with a focus on current open problems. You will be made aware of these open problems during practical sessions that will cover active research in a major Bioinformatics problem (mutation impact prediction). Based on your background, you will be assigned to separate classes where you will be working to fill gaps in your background knowledge in programming and/or biology.

After completing the course successfully:

- You can explain the fundamental principles of cell biology and the genetic information flow
- You can write short code to solve simple problems and complete skeleton scripts for biological applications
- You can read algebra and calculus equations
- You can become aware of gaps in your own background knowledge.
- You can address gaps in your background knowledge (programming, mathematics and molecular biology) and skills
- You are aware of the major issues when developing and benchmarking new methodology in bioinformatics research
- You can present and communicate a joined group projects carried out with students of diverse backgrounds (Interdisciplinarity) in a respectful way
- You can apply and understand fundamental methods in bioinformatics, e.g. sequence alignment, impact prediction, and to understand and use fundamental concepts in bioinformatics, e.g. homology, evolutionary

conservation, and gene function

Fundamental knowledge:

Students will have the opportunity to level up their knowledge of molecular biology, maths (linear algebra and calculus) and programming (Python), according to their needs.

Course Content

Topics:

- Evolution, genomics, sequence alignment, Blast/PSI-Blast, FAIR data, benchmarking, machine learning, genome assembly, mutation impact prediction, protein stucture analysis, substitution matrices, scientific writing

Project:

- Group project to benchmark different mutation impact prediction approaches. The groups will be composed to include different BSc backgrounds.

Background knowledge (1 out of 3, see below):

- Molecular cell biology
- Programming and computational thinking in Python
- Calculus and linear algebra

Additional Information Teaching Methods

- 8 lectures (two-hour lecture in the morning, two days per week) Exercises during/in between lectures
- 12 computer practicals for the group project - compulsory attendance (online or on campus) (two hour sessions following the morning lectures, two days per week), supervised.
- Feedback (theoretical and practical) will be given during the computer practical sessions.

Conversion classes for background knowledge:

- Molecular cell biology (7 x double lectures)
- Programming and computational thinking in Python (10 computer practicals)

Depending on your background, you will be assigned 1 of the 2 conversion classes. Students that are already well versed in both topics (molecular biology and programming) may be assigned to the following class, which is organised as part of the Introduction to Systems Biology:

- Calculus and linear algebra (10 maths classes)

Method of Assessment

- Project work [~40%] :
- Project report and/or presentation
- Digital exam to assess [~60%]:
- Lecture topics
- Concept learned during the project
- Concepts from conversion classes

(assessment of practical assignments may be adjusted based on ongoing developments in generative language models)

Further assessment and grading details will be posted on Canvas (resits and compensation rules)

Entry Requirements

Bachelor in any science discipline (including medicine) or strong programming background, and an interest in biological problems.

Literature

- course material (slides, scientific papers) on canvas.vu.nl
- book: Marketa Zvelebil and Jeremy O. Baum Understanding Bioinformatics Garland Science 2008 ISBN-10: 0-8153-4024-9

Additional Information Target Audience

M Artificial Intelligence
M Bioinformatics and Systems Biology
M Biomolecular Sciences
M Oncology
M Computational Science

Additional Information

BYOD policy (Bring Your Own Device)
We expect students in this course to use their own laptop. This laptop should at the very least support an SSH client, for remote shell access to the VU Linux servers. We will help you to set this up.

Recommended background knowledge

Programming in Python, Molecular Biology, some basics of calculus and linear algebra

High Performance Computing and Big Data

Course Code	XMU_40013
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130398

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Machine Learning for the Quantified Self

Course Code	XM_40012
Credits	6
Period	P6
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	V.G.M. Francois Lavet, prof. dr. M. Hoogendoorn, B. Borsboom

Teaching method(s)	Practical, Lecture

Course Objective

The main aims of this course are to make the student familiar with specific machine learning techniques for quantified self and sensory data. Both the theory as well as the practical side of matters will be addressed. More specifically, the following aims are distinguished:

1. Understanding the domain of the quantified self and sensory data
2. Knowing and being able to apply techniques for outlier detection on sensory data.
3. Knowing and being able to apply techniques for feature engineering on sensory data.
4. Knowing and being able to apply clustering techniques related to sensory data.
5. Understanding the theory of supervised learning and its implications.
6. Learning how to apply non-temporal machine learning techniques to sensory data.
7. Knowing and being able to apply temporal machine learning techniques to sensory data, including deep learning techniques.
8. Knowing and being able to apply reinforcement learning in the application of the quantified self.

The course if focused on the Dublin descriptors Knowledge and understanding (learning new relevant algorithm in this domain); Applying Knowledge and Understanding (since the aim is also to apply the algorithms in practice); Making judgements (which technique is appropriate, how to best apply the techniques for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

Course Content

The quantified-self refers to large-scale data collection of a user's behavior and context via a range of sensory devices, including smart phones, smart watches, ambient sensors, etc. These measurements contain a wealth of information that can be extracted by means of machine learning techniques, for instance for the purpose of predictive modeling. In addition, machine learning techniques can be a driver for adaptive systems to support users in a personalized way based on the aforementioned measurements. The type of data does however require specialized machine learning techniques to fully exploit the information contained in the data. Examples of challenges include the temporal nature of the data, the variety in the type of data, the different granularity of various sensors, noise, etcetera.

In this course specific techniques to handle quantified self (or broader sensory data) will be treated. More in specific, it will address:

- Feature engineering (how do we come from raw data to usable features):
  * Removing noise from data
  * Handling missing data
  * Identifying (temporal) features
- Learning of user patterns:
  * Temporal machine learning approaches such as recurrent neural networks, time series analysis
  * Clustering approaches with dedicated distance metrics (including dynamic time warping)
- Adaptive feedback and support
  * Reinforcement learning
- Integration of the various components.

In addition, a number of real-life applications will be discussed. Next to lectures, there will be an extensive practical part, where students will learn to work with various algorithms work on a project they propose themselves.

Additional Information Teaching Methods

The course will be taught in four weeks. Both regular lectures (l, first two weeks) and practical sessions (p) will be organized. During the first two weeks the emphasis will be on the lectures and getting started with the practical assignment that ties closely to the content of the lectures. The third week will be working on the practical assignment fulltime. In the fourth week a mandatory final lecture will take place in which a selection of groups will present their project. The course ends with an exam.

Method of Assessment

Written exam (50%) and practical assignment (50%). For both parts, the grade needs to be sufficient to obtain a final grade. The practical assignment is done in groups of three. For the practical assignment, two intermediate versions of the report should be submitted for feedback that are given a pass/fail. Only in case of a pass for the two intermediate versions will a final grade be given for the practical assignment.

The written exam will be multiple choice and have a resit during the Summer period. On top, in case your

assignment grade average is insufficient you are allowed to redo the assignments by improving it based on the feedback you received. The maximum grade for this resubmitted assignment is set to a 6.

Literature

Hoogendoorn, M. and Funk, B. Machine Learning for the Quantified Self - On the Art of Learning from Sensory Data, ISBN 978-3-319-66308-1, Springer Verlag 2018. The book can be downloaded free of charge through the UBVU, just follow the link to the book: http://dx.doi.org/10.1007/978-3-319-66308-1

Papers on state-of-the-art deep learning techniques.

Additional Information Target Audience

Master Artificial Intelligence, Master Business Analytics, Master Computer Science

Additional Information

Lecturer: Prof.dr. M. Hoogendoorn

Custom Course Registration

Registration for medical informatics (UvA) students: Please check the following document https://www.dropbox.com/s/a2kd7xacuxt5y8e/AI%20for%20Health%202022-2023.
and then register for secondary courses via https://vu.nl/en/student/information-about-registration-and-enrolment/in

Recommended background knowledge

Programming experience. Knowledge of basic machine learning algorithms.

Parallel Programming Practical

Course Code	X_400162
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	mr. M. Cassar, drs. C. Verstoep, dr. F. Verdugo Rojano
Teaching method(s)	Lecture

Course Objective

At the end of the course, the students will:

- Be able to apply distributed parallel programming models to implement algorithms on large-scale computing systems. (Applying knowledge and understanding)
- Have hands-on experience in executing and testing parallel programs using a cluster computing environment. (Applying knowledge and understanding, making judgments)
- Be able to critically evaluate the performance of a distributed parallel program, identifying bottlenecks, and perform optimizations. (Applying knowledge and understanding, making judgments)
- Effectively communicate parallelization approaches and performance results by writing scientific reports. (Communication, lifelong learning skills)

Course Content

This course is a follow-up of "Programming Large-Scale Parallel Systems", which allows students to apply the knowledge studied in "Programming Large-Scale Parallel Systems" in a more practical, hands-on setting. Throughout the course, students will transform given sequential algorithms into parallel programs able to run on multiple compute nodes within a large-scale computing system. We will consider distributed programming models, including message-passing with MPI and the task-oriented model based on remote procedure calls included in the Julia programming language. The programs will be tested for correctness and benchmarked for performance on the DAS-5 cluster computer (see http://www.cs.vu.nl/das5). The performance of the program will be measured, analyzed and, whenever necessary, optimized. Profiling tools will be considered to identify bottlenecks and to guide

optimization. A scientific report must be written for each problem discussing the parallelization approach, the implementation choices, and the outcomes of the performance analysis.

Additional Information Teaching Methods

- A kick-off meeting to introduce the course objectives and the practical assignments.
- After the kick-off meeting, students work independently towards the completion of the assignments.
- There are weekly meetings for Q&A and to guide students in different aspects of the work.

Method of Assessment

- (90%) Practical assignments that include practical programming work and written report.
- (10%) Individual oral exam, where students defend their work in a presentation about the assignments and by answering questions from the teachers.

Additional rules

- The grade of the practical assignments should be at least 5.5 to pass the course.
- The grade of the individual oral exam should be at least 5.5 to pass the course.
- There is no resit opportunity for the practical assignments nor the oral exam.

Literature

All course materials, including documentation and reference materials, will be made available through Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Computational Science

Recommended background knowledge

- We highly recommend having attended "Programming Large-Scale Parallel Systems" before joining "Parallel Programming Practical".
- The algorithms will be implemented in the Julia programming language. An introduction to parallel computing in Julia is given in the course "Programming Large-Scale Parallel Systems". Otherwise, students can study the lecture notes in https://www.francescverdugo.com/XM_40017/dev/.

Performance of Networked Systems

Course Code	X_405105
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. R.D. van der Mei
Examiner	prof. dr. R.D. van der Mei
Teaching Staff	prof. dr. R.D. van der Mei
Teaching method(s)	Lecture

Course Objective

Students will acquire basic knowledge of:

- Performance issues in communication systems, a complex interplay between communication networks (TCP/IP, UDP), application servers (HTTP, QUIC), services and clients
- Performance models for wireless and mobile network protocols (WiFi, GSM, GPRS, UMTS, HSDPA, 4G, 5G, 6G and mobile ad-hoc networks)
- Performance engineering principles and management of traffic in IP-based networks
- Performance modeling of streaming and elastic traffic
- Quantitative models for planning capacity in communication networks

Students will gain experience in modelling, engineering and planning performance of networked systems, and will

learn how to tackle performance problems arising in today's information and communication systems.

Course Content

Over the past few decades, information and communication technology (ICT) has become ubiquitous and globally interconnected. As a consequence, our ICT systems are expected to process huge amounts of digital information, which puts a tremendous burden on our ICT infrastructure. At the same time, our modern society has become largely dependent on the well-functioning of our ICT systems: large-scale system failures and perceivable Quality of Service (QoS) degradation may completely disrupt our daily lives and have huge impact on our economy. These developments raise the need for understanding the nature and source of performance problems, and for finding effective ways to make our networks robust and reliable.

Motivated by this, the course will address performance-related challenges and solution methods in communication networks. To this end, we focus on performance modeling, traffic management, QoS provisioning and capacity planning for both wired and wireless/mobile communication networks (such as WiFi, 2G, 3G, 4G, 5G, 6G networks, and mobile ad-hoc networks). We address questions like:

- How do wired and wireless communication network technologies work?
- How can we model their performance?
- How can we manage and control traffic flows and provision QoS in IP-based networks?
- How can we deal with the omnipresence of uncertainty in virtually all aspects of communication networks?
- What techniques are available to model, analyze and optimize performance?

Additional Information Teaching Methods

Classroom lectures and a practical take-home homework assignment.

Method of Assessment

The assessment will be based on the homework assignment (total 20%) and a written exam (80%). Homework assignments can be made in groups of one or two students.

Literature

- A textbook on "High Performance Browser Networking" by Ilya Grigorik, O-Reilly, 2013.
- A reader on Stochastic Performance Modelling.
- Individual papers on specific topics.

Additional Information Target Audience

mBA, mCS, mPDCS, mCLS, mEctrie

Explanation Canvas

The relevant course information will be shared via CANVAS.

Recommended background knowledge

The students should have basic knowledge of computer networks.

Programming Large-Scale Parallel Systems

Course Code	XM_40017
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	dr. F. Verdugo Rojano, mr. M. Cassar
Teaching method(s)	Lecture

Course Objective

At the end of the course, students will be able to

- Understand the main principles of High Performance Computing (Knowledge and understanding)
- Design and implement efficient parallel algorithms on distributed-memory parallel computers (Applying knowledge and understanding)
- Make use of different distributed parallel programming models to implement these algorithms, including message passing with MPI and the task-based model based on remote procedure calls available in the Julia programming language (Applying knowledge and understanding).
- Analyze the performance of parallel applications and network topologies (Making judgements)
- Identify the main parallelization challenges in selected scientific computing applications (Knowledge and understanding)

Course Content

In this course, students learn how to design and implement computer programs that solve a problem in parallel on a distributed-memory computing system such as the supercomputers available in the top500 list (https://top500.org/). This is key in many real world applications to reduce execution time and to solve very large problems that cannot fit in a single compute node. You will learn how to design such parallel algorithms in order to minimize different types of performance overheads such as communication overhead, search overhead, and load imbalance. You will also learn how to analyze the theoretical and experimental performance of the resulting codes and the underlying computer network topology. Among all parallel programming models available for high performance computing, this course focuses on distributed-memory parallelization. We consider different distributed parallel programming models, including established techniques such as message-passing with MPI, but also modern approaches like the task-based model available in the Julia programming language. The course also discusses the application of these techniques to a set of real-world scientific computing applications, including partial-differential equations in solid and fluid dynamics, and n-body problems in astrophysics.

Additional Information Teaching Methods

- There are 2 class sessions per week. Part of the course follows the flipped classroom method. Pre-recordings, lecture notes, and exercises are made available in advance for self study, and follow up discussions take place in class meetings to deepen understanding.
- The course also includes the opportunity to run experiments on the DAS-5 computer cluster (http://www.cs.vu.nl/das5).
- Computer codes used in the course are implemented in the Julia programming language. Previous knowledge of Julia is not required as it will be learned as part of the course.

Method of Assessment

There is a final written exam (E) and a programming assignment (A). Their grades are combined as G = 0.1*A + W(E)*E, where

- W(E) = 1 for E < 7, and
- W(E) = 1-(E-7)/30 otherwise.

The grades E and A are at most 10 points each.

According to this formula the programming assignment is "quasi optional". There is little or no penalty for skipping the assignment in most of the cases. However, high-scoring students in the exam need the assignment for a perfect score.

Literature

Course material will be made available on Canvas. Part of the course is based on the lecture notes available at https://www.francescverdugo.com/XM_40017/dev/.

Additional Information Target Audience

Master Business Analytics
Master Computational Science
Master Computer Science
Master Computer Security

Additional Information

There is also a separate course "Parallel Programming Practical" (6 EC) in Period 3, the aim of which is to complement the contents of this course with further practical skills and experience.